UniFECAF

Object Oriented Programming

Sistema Automotivo
Gestão de Estoque de Veículos

Relatório prático — Evidências e códigos principais

Trabalho acadêmico de programação orientada a objetos.
Java 21 · Spring Boot 4.1.2 · MySQL 8.4

31 de outubro de 2026

1. Implementação e ambiente

API REST de gestão de estoque de veículos, com cadastro de marcas e modelos, CRUD de veículos e filtros. Java 21.0.12.1, Spring Boot 4.1.2 e MySQL Community Server 8.4.13. Execução local em Windows com banco MySQL em container Docker. Nas capturas, a API está na porta 8081; o padrão configurado para reprodução é 8080.

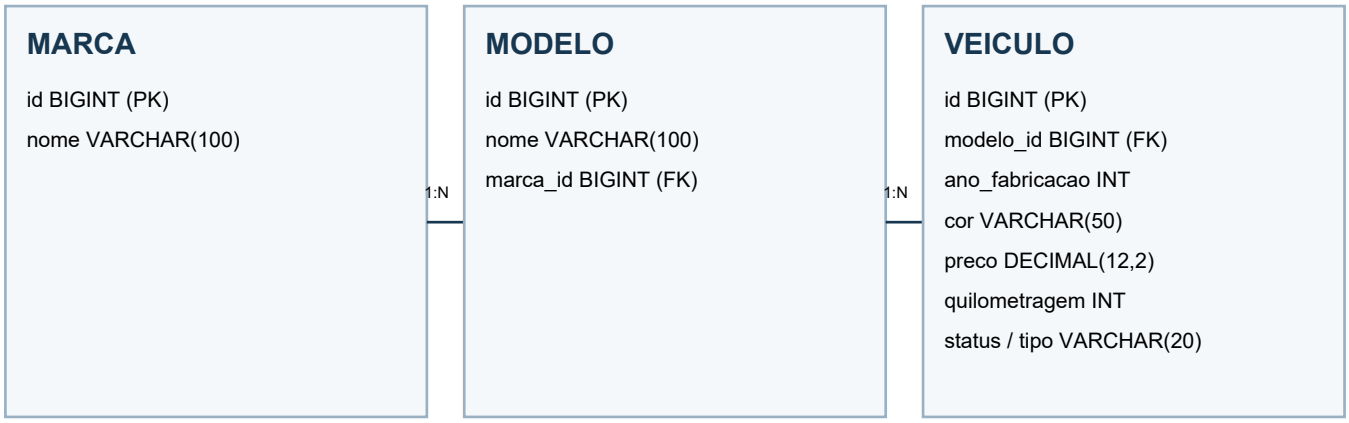
O cliente HTTP local utilizado nas capturas e no vídeo envia requisições reais para a API. Ele é uma ferramenta de demonstração, não um frontend do sistema. O frontend é opcional no enunciado. As figuras preservam as respostas efetivamente retornadas.

Repositório público: https://github.com/igorguimaraes-dev/sistema-automotivo-poo.

2. Estrutura e arquitetura

Cliente HTTP → Controller → Service → Repository → MySQL. Controllers definem rotas REST e validam os contratos de entrada. Services executam regras de negócio e transações. Repositories persistem e consultam as entidades por JPA. DTOs representam entradas/saídas; TratamentoErros padroniza respostas de validação e conflito.

3. Modelo físico



O arquivo database/schema.sql define as tabelas e suas PKs/FKs, restrições de preço, quilometragem, status e tipo, além de índices. A marca do veículo é obtida por seu modelo. O Hibernate valida a compatibilidade do esquema; não recria os dados a cada início.

4. Resultados dos testes

Foi executado o comando mvnw.cmd clean verify no JDK 21, com MySQL real e um banco separado chamado estoque_veiculos_test. Resultado: **4 cenários, 0 falhas, 0 erros, 0 ignorados**. Os cenários incluem várias requisições e verificações de dados, não apenas quatro endpoints.

Cenário de integração	Resultado
Cadastro, consulta, atualização e exclusão com verificação SQL	Aprovado
Filtros por marca/modelo/preço/ano/status/tipo e combinação	Aprovado
Entradas inválidas não criam registros nem alteram veículo válido	Aprovado
Associações inexistentes, duplicidades e política de exclusão	Aprovado

A persistência após reinício também foi comprovada manualmente: a aplicação de PID 15440 foi encerrada e reiniciada a partir do JAR com PID 10400. O veículo de ID 1 continuou com preço 108000,50, quilometragem 36000 e status VENDIDO. A captura correspondente aparece na sequência de evidências.

5. Performance observada

Medições locais por HTTP, com a API e o MySQL na mesma máquina e dois veículos na base de medição, após a exclusão demonstrada. Para cada operação, foi descartada uma chamada de aquecimento e registradas cinco amostras. Esses dados não representam teste de carga nem garantem o mesmo tempo em outro ambiente.

Operação	Média (ms)	Maior tempo (ms)
Consulta geral	55.54	59.22
Filtro combinado	55.55	68.99
Atualização	34.88	55.13

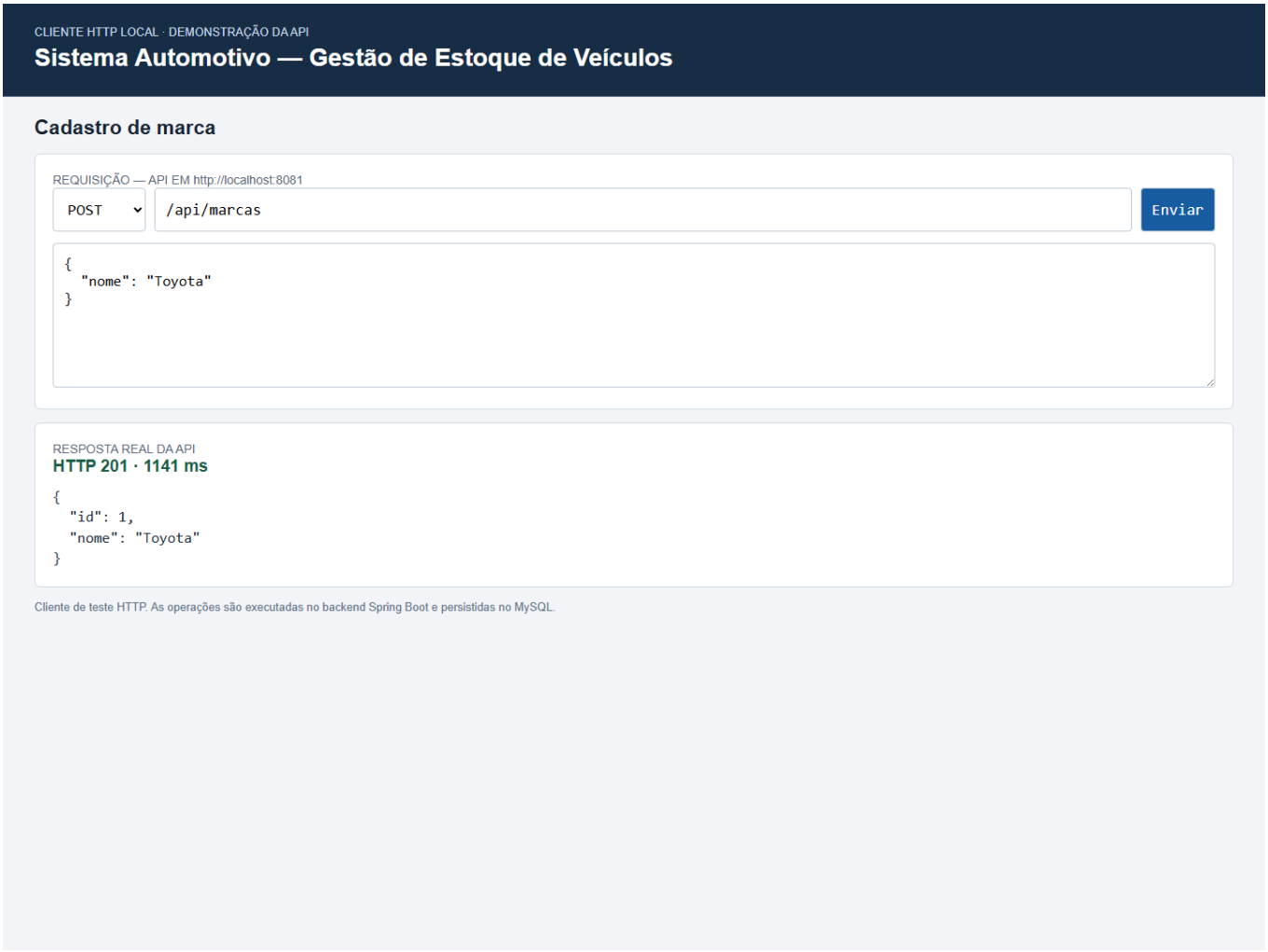
As consultas aplicam os filtros no banco, sem carregar todo o estoque para filtrar em Java. Os resultados retornam listas vazias quando não há correspondências. Entradas inválidas retornam 400; IDs inexistentes, 404; conflitos de regra ou duplicidade, 409.

6. Capturas do sistema em execução

Cada figura mostra o método, a rota, os dados enviados quando aplicável, o código HTTP e a resposta real. Foram cadastrados veículos Toyota/Corolla, Fiat/Argo e Honda/CG 160 para verificar a seleção e a exclusão de registros pelos filtros.

Figura 1 — Cadastro de marca

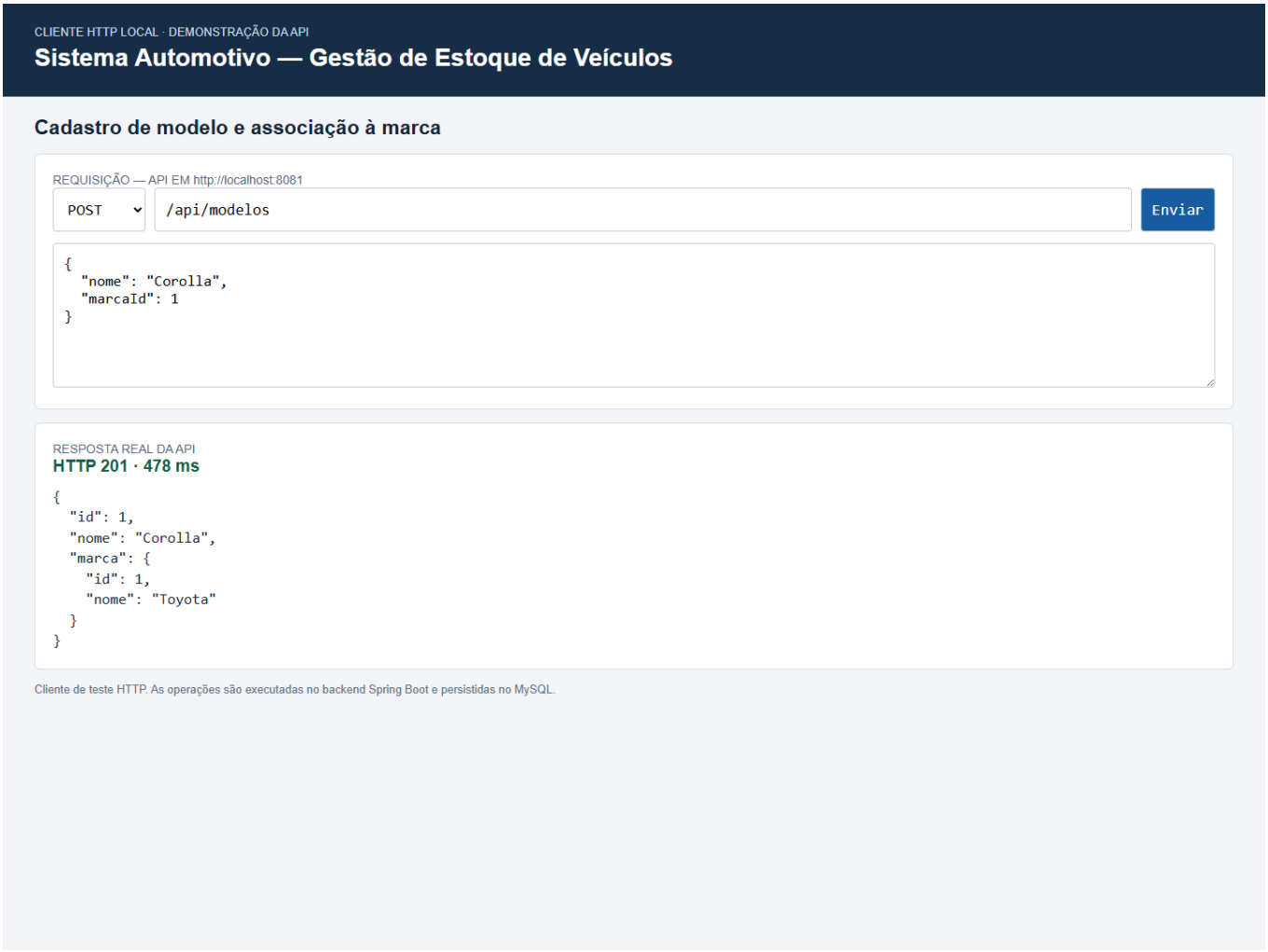
POST /api/marcas
Resultado observado: HTTP 201.



Fonte: captura da execução local da API por cliente HTTP.

Figura 2 — Cadastro de modelo e associação à marca

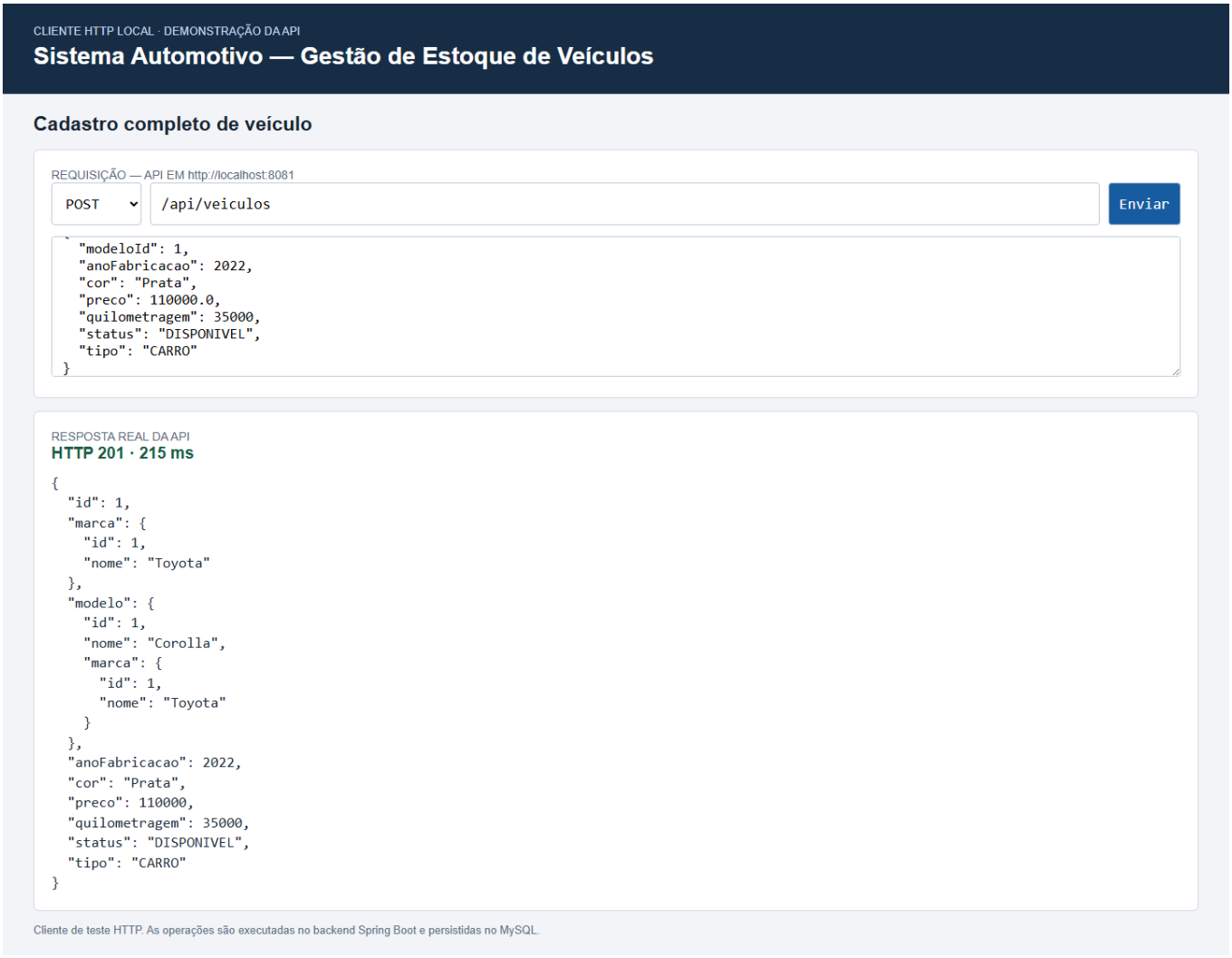
POST /api/modelos
Resultado observado: HTTP 201.



Fonte: captura da execução local da API por cliente HTTP.

Figura 3 — Cadastro completo de veículo

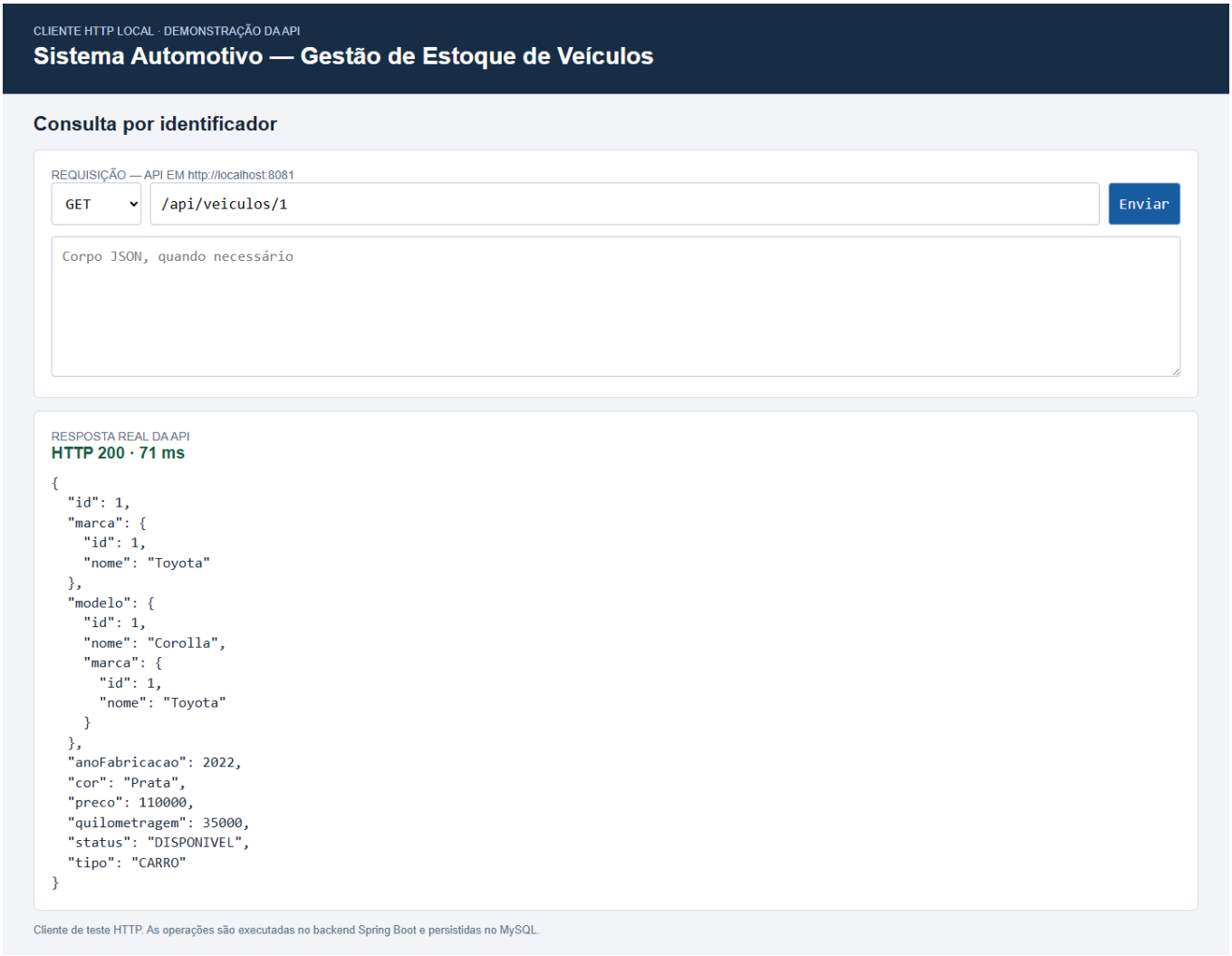
POST /api/veiculos
Resultado observado: HTTP 201.



Fonte: captura da execução local da API por cliente HTTP.

Figura 4 — Consulta por identificador

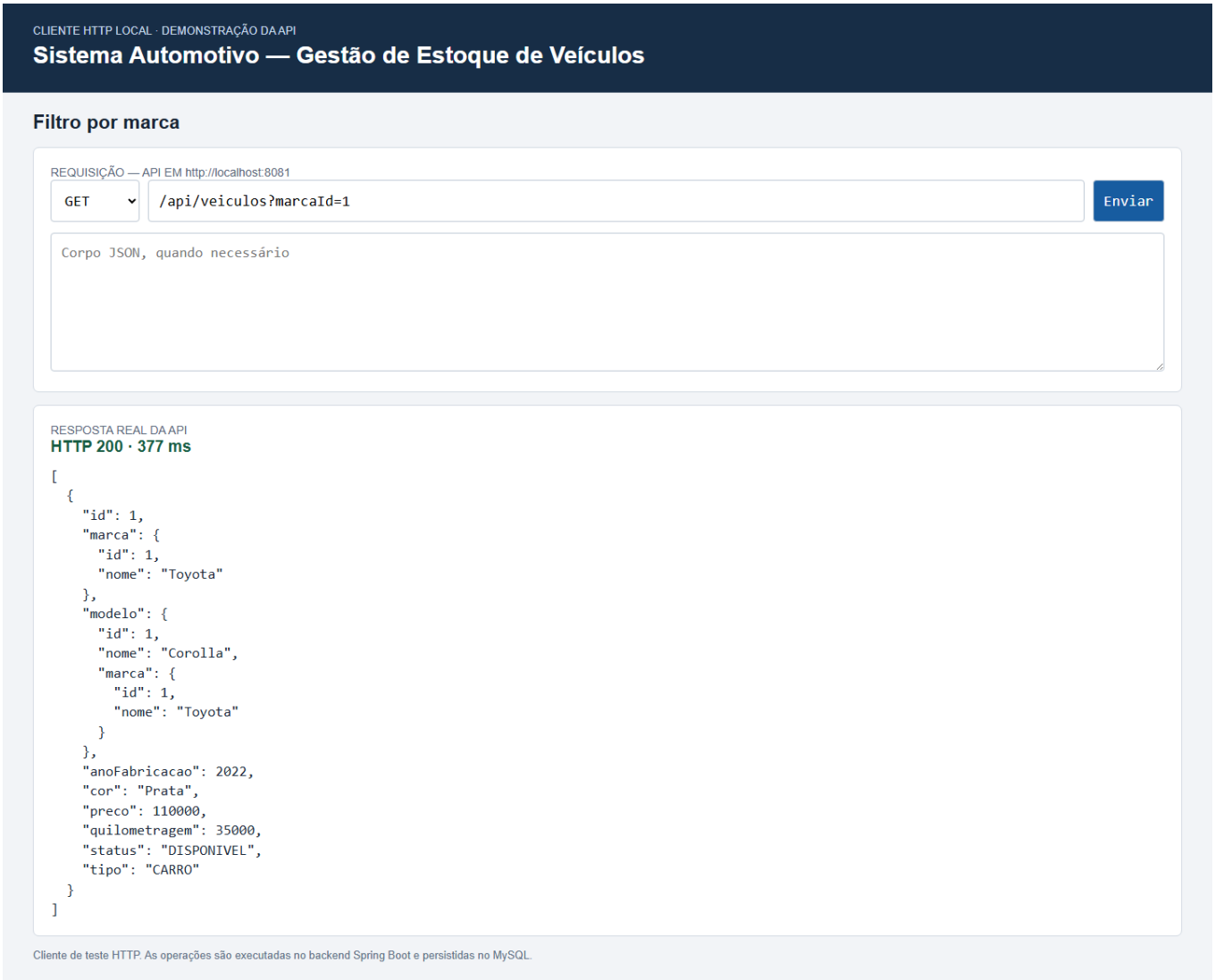
GET /api/veiculos/1
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 5 — Filtro por marca

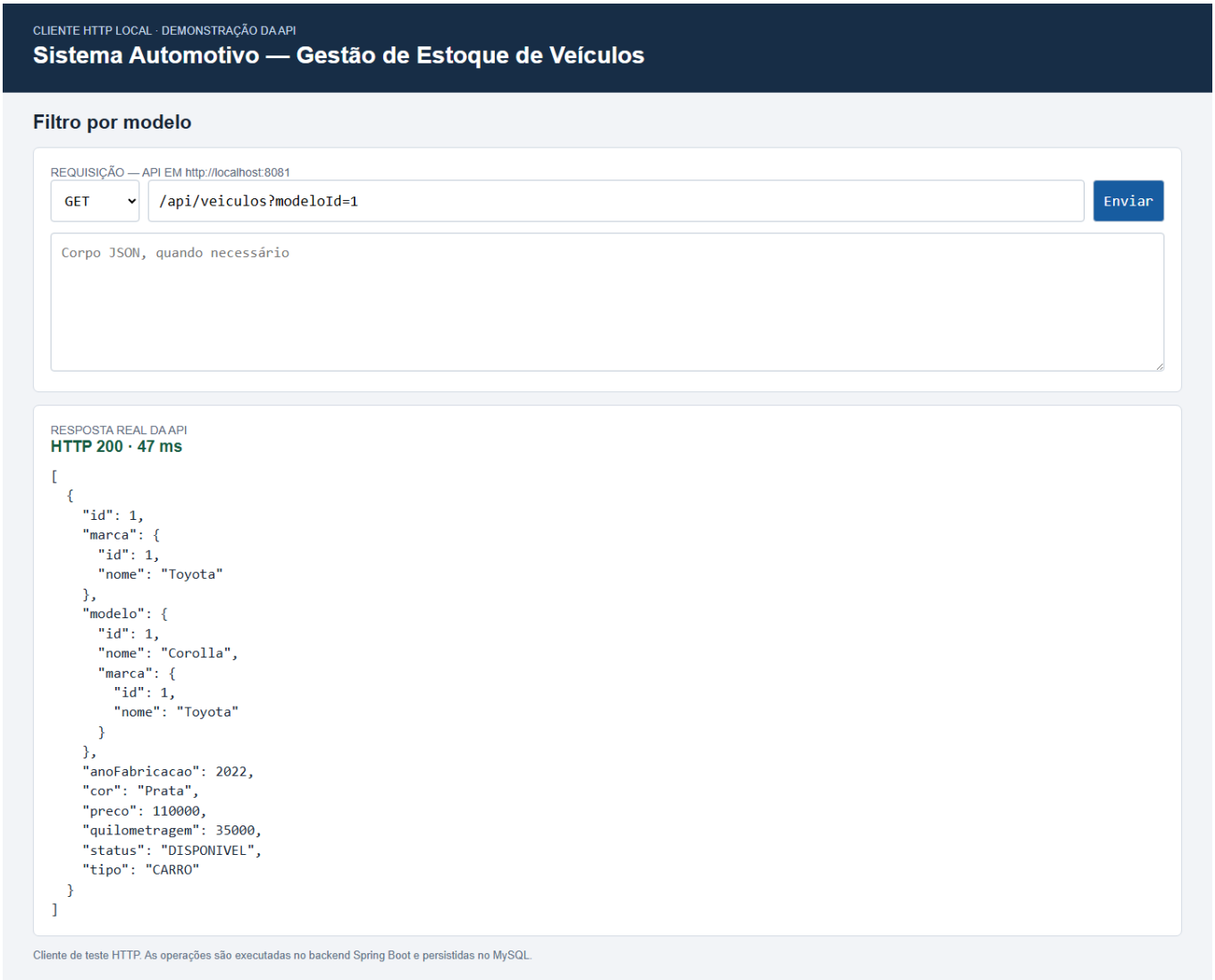
GET /api/veiculos?marcaId=1
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 6 — Filtro por modelo

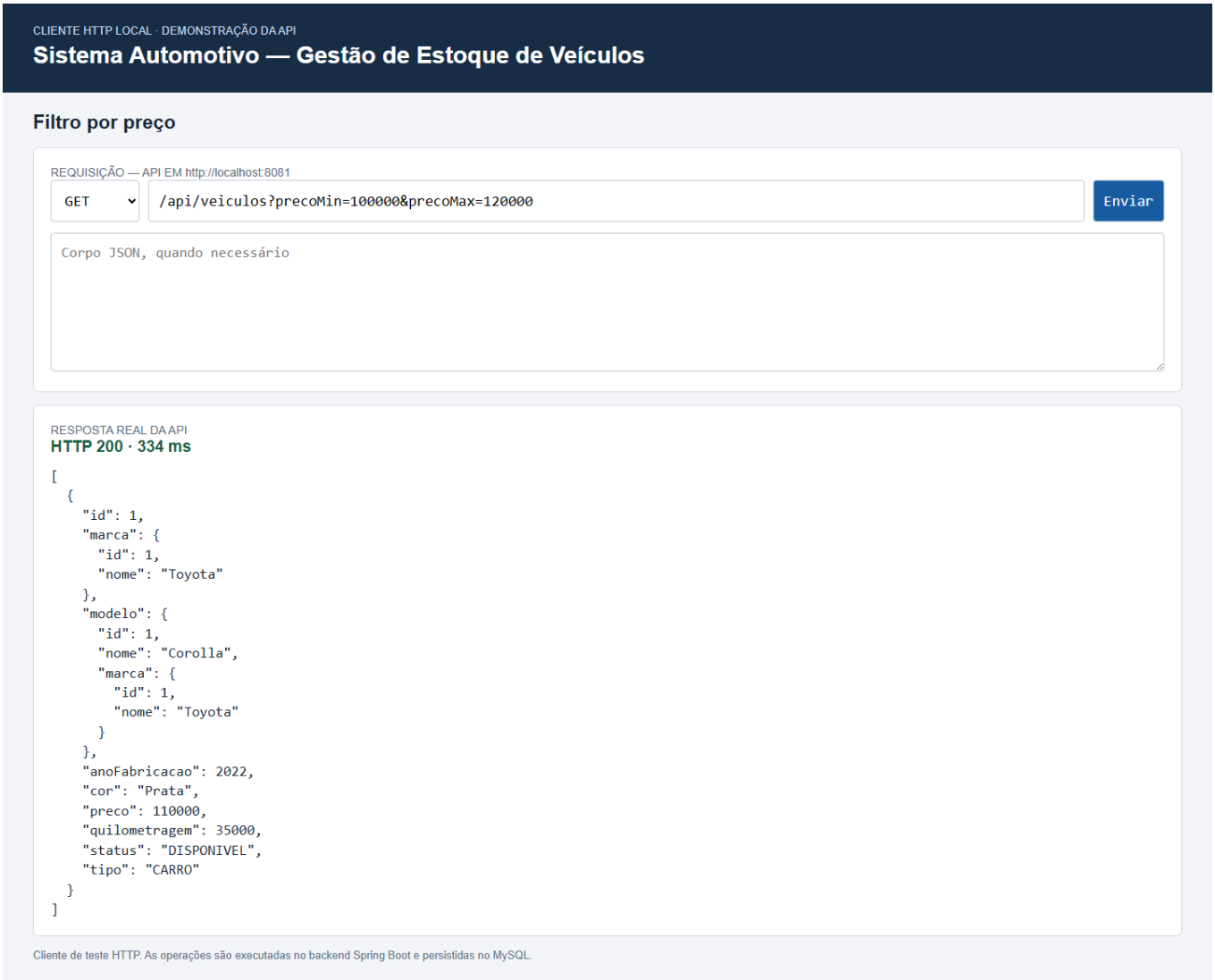
GET /api/veiculos?modeloId=1
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 7 — Filtro por preço

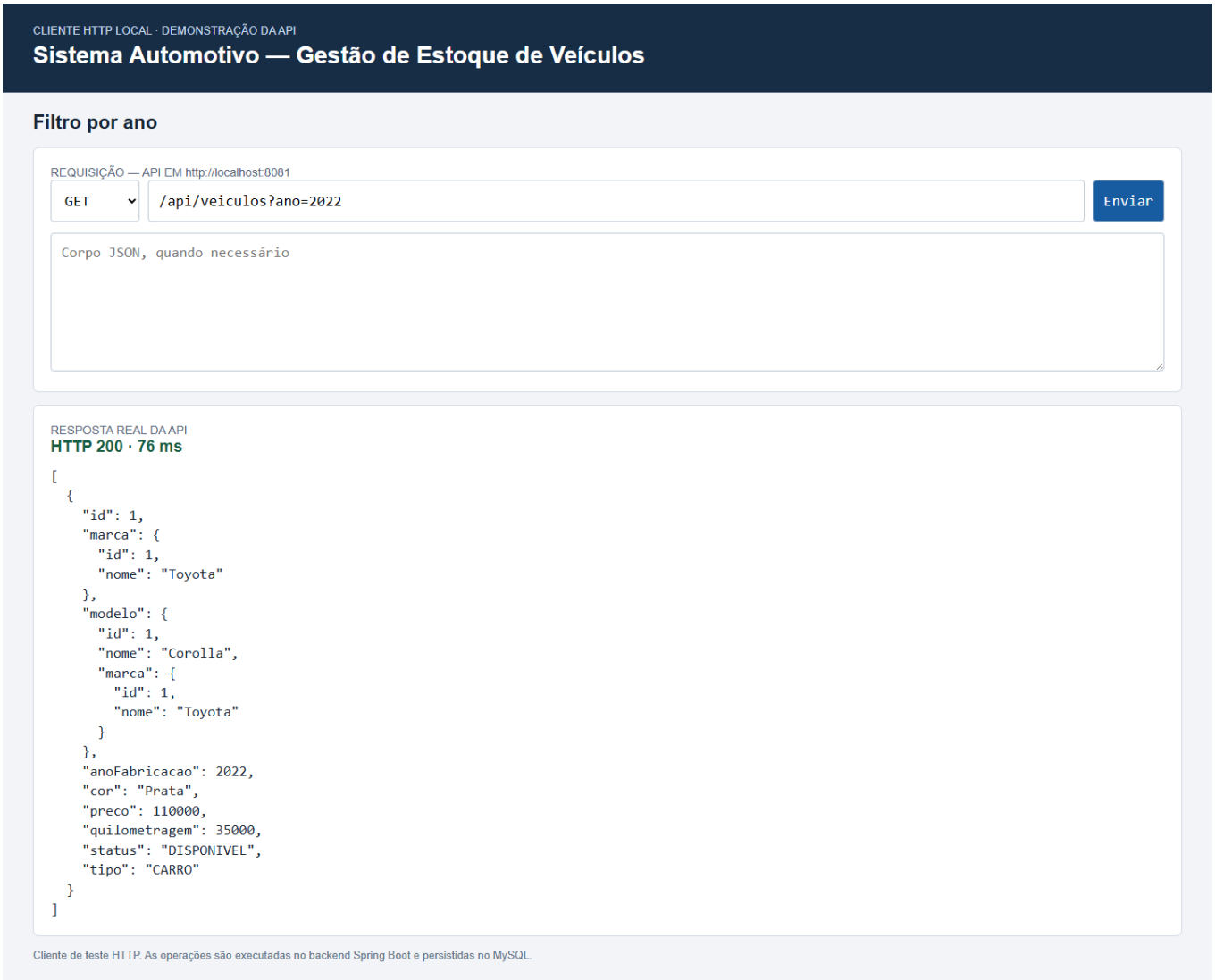
GET /api/veiculos?precoMin=100000&precoMax=120000
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 8 — Filtro por ano

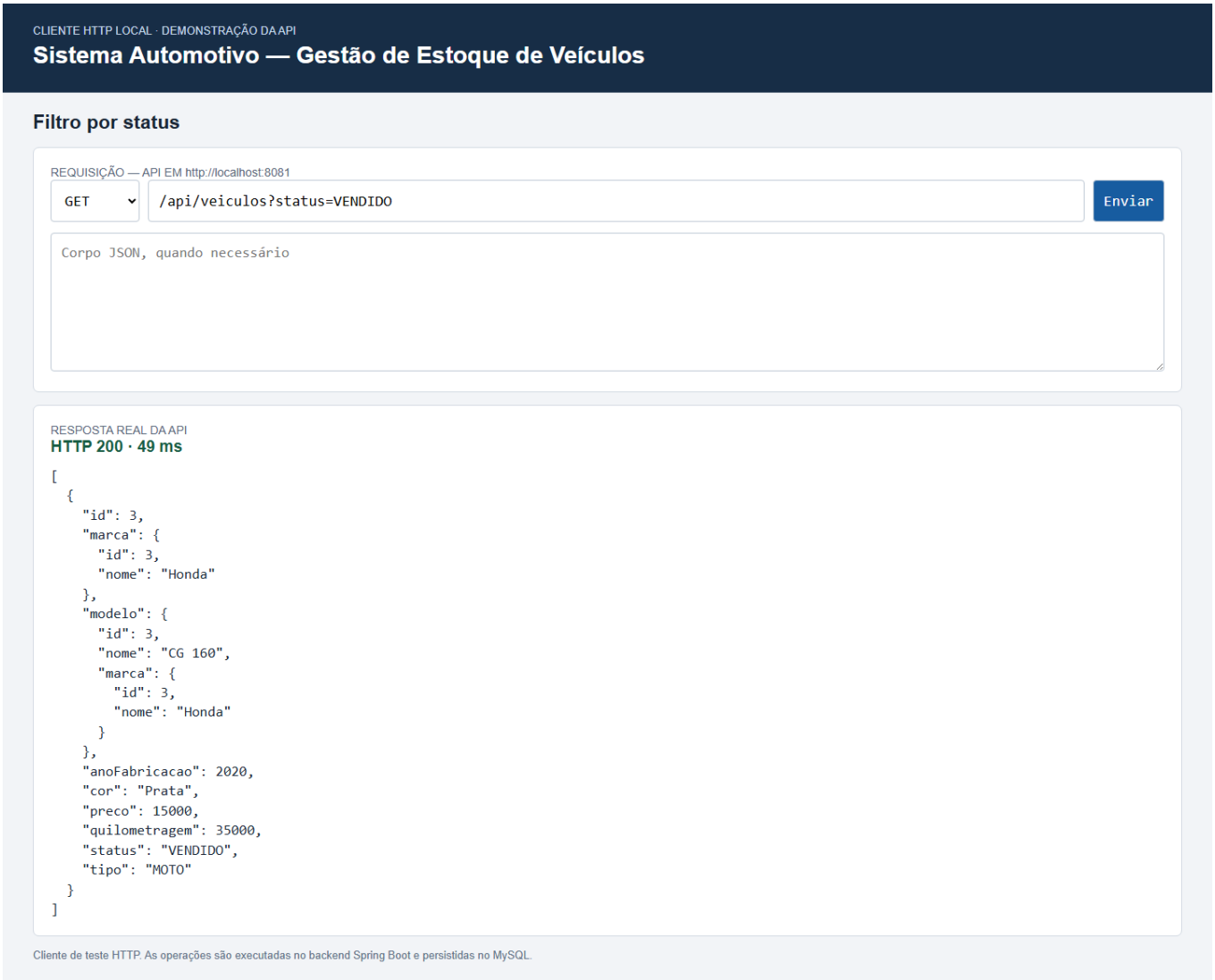
GET /api/veiculos?ano=2022
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 9 — Filtro por status

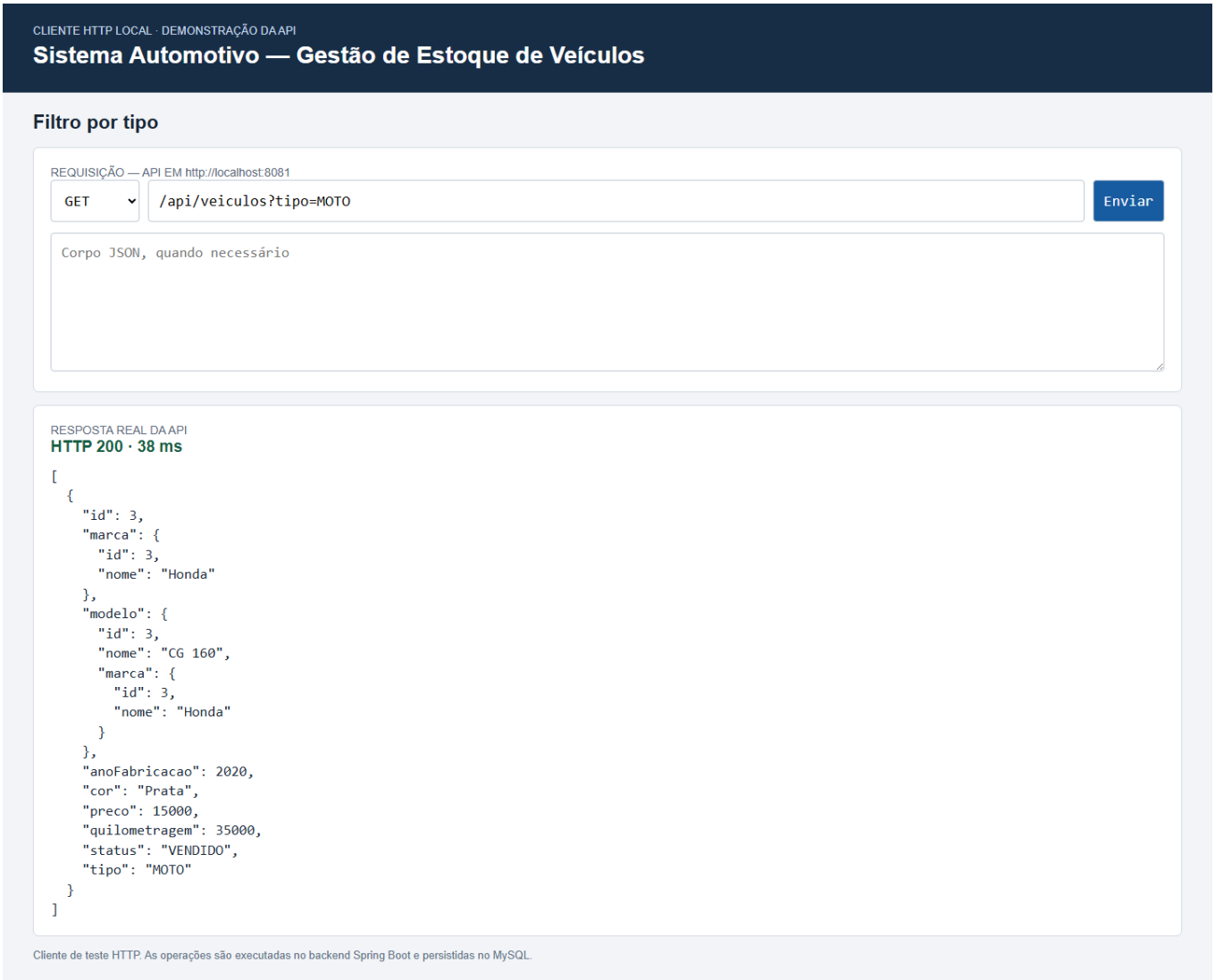
GET /api/veiculos?status=VENDIDO
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 10 — Filtro por tipo

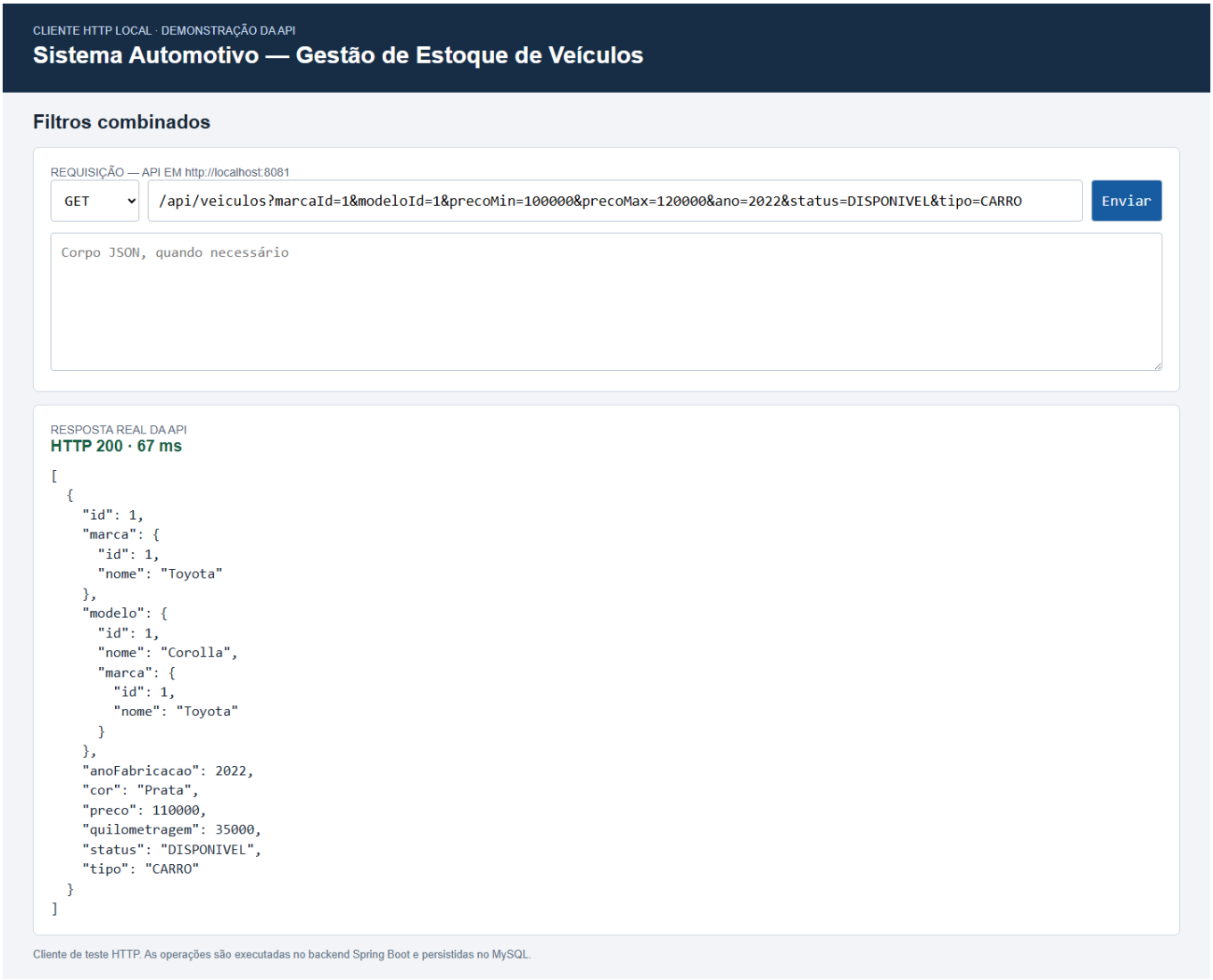
GET /api/veiculos?tipo=MOTO
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 11 — Filtros combinados

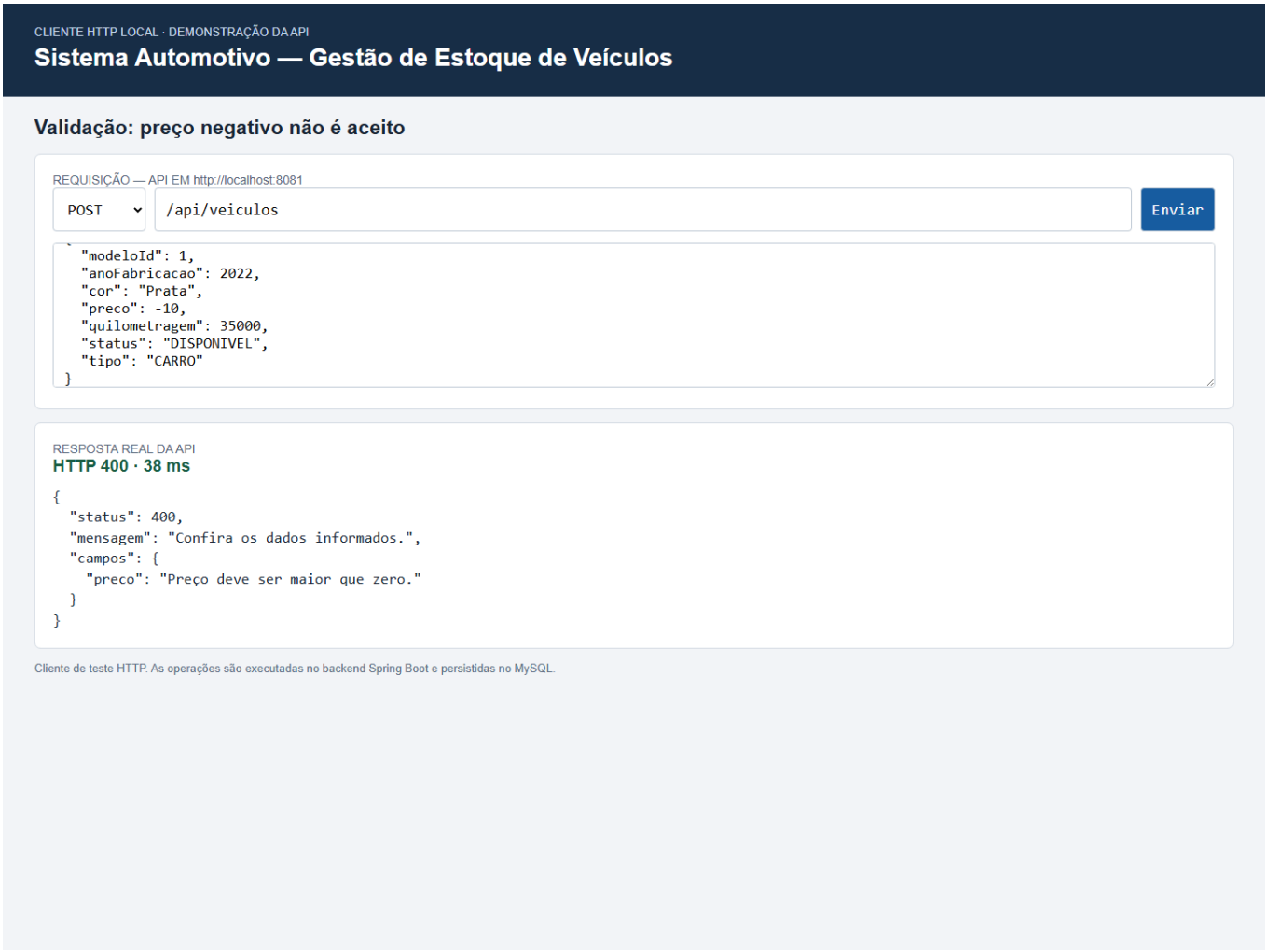
GET /api/veiculos?marcaId=1&modeloId=1&precoMin=100000&precoMax=120000&ano=2022&status=DISPONIVEL&tipo=CARRO
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 12 — Validação: preço negativo não é aceito

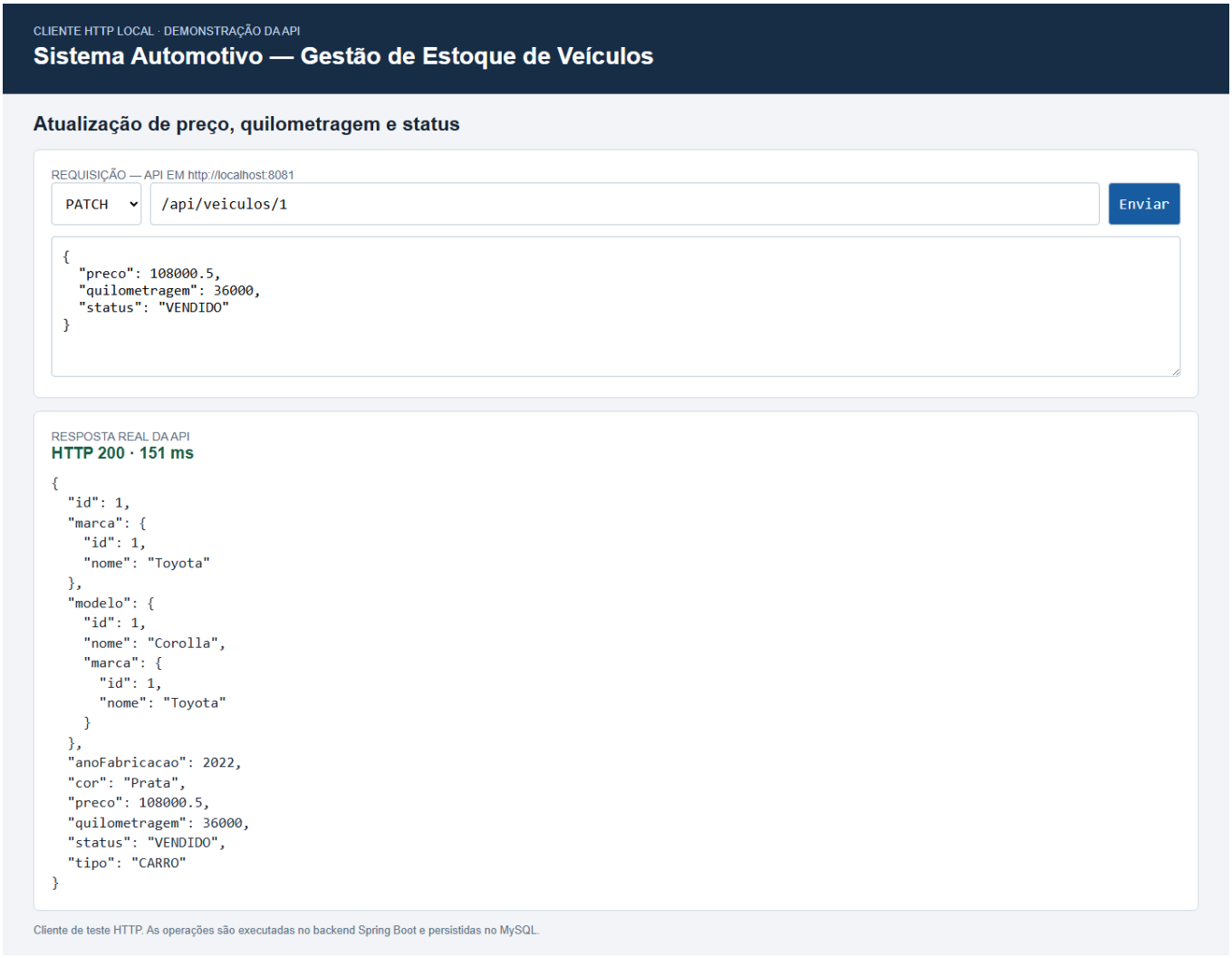
POST /api/veiculos
Resultado observado: HTTP 400.



Fonte: captura da execução local da API por cliente HTTP.

Figura 13 — Atualização de preço, quilometragem e status

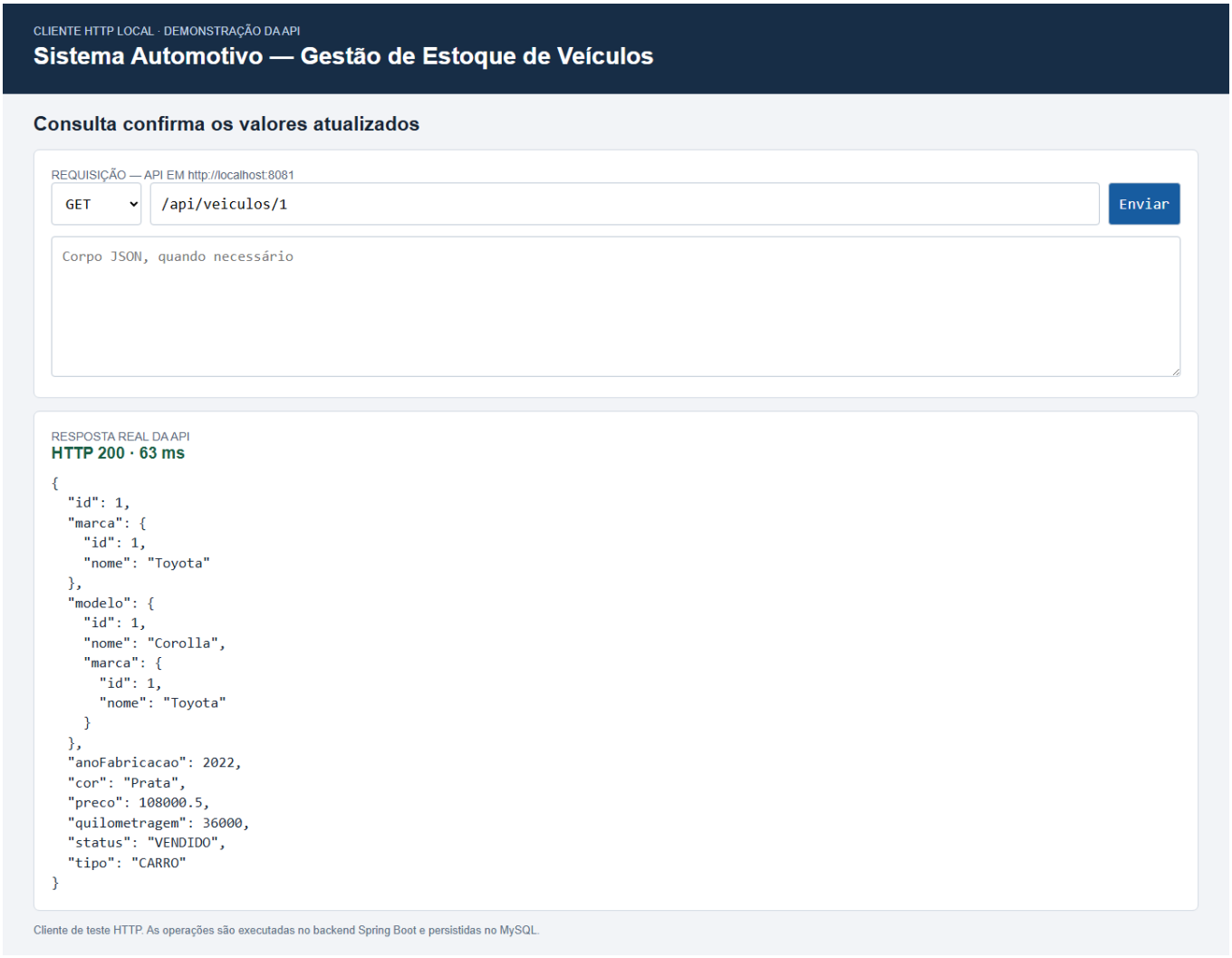
PATCH /api/veiculos/1
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 14 — Consulta confirma os valores atualizados

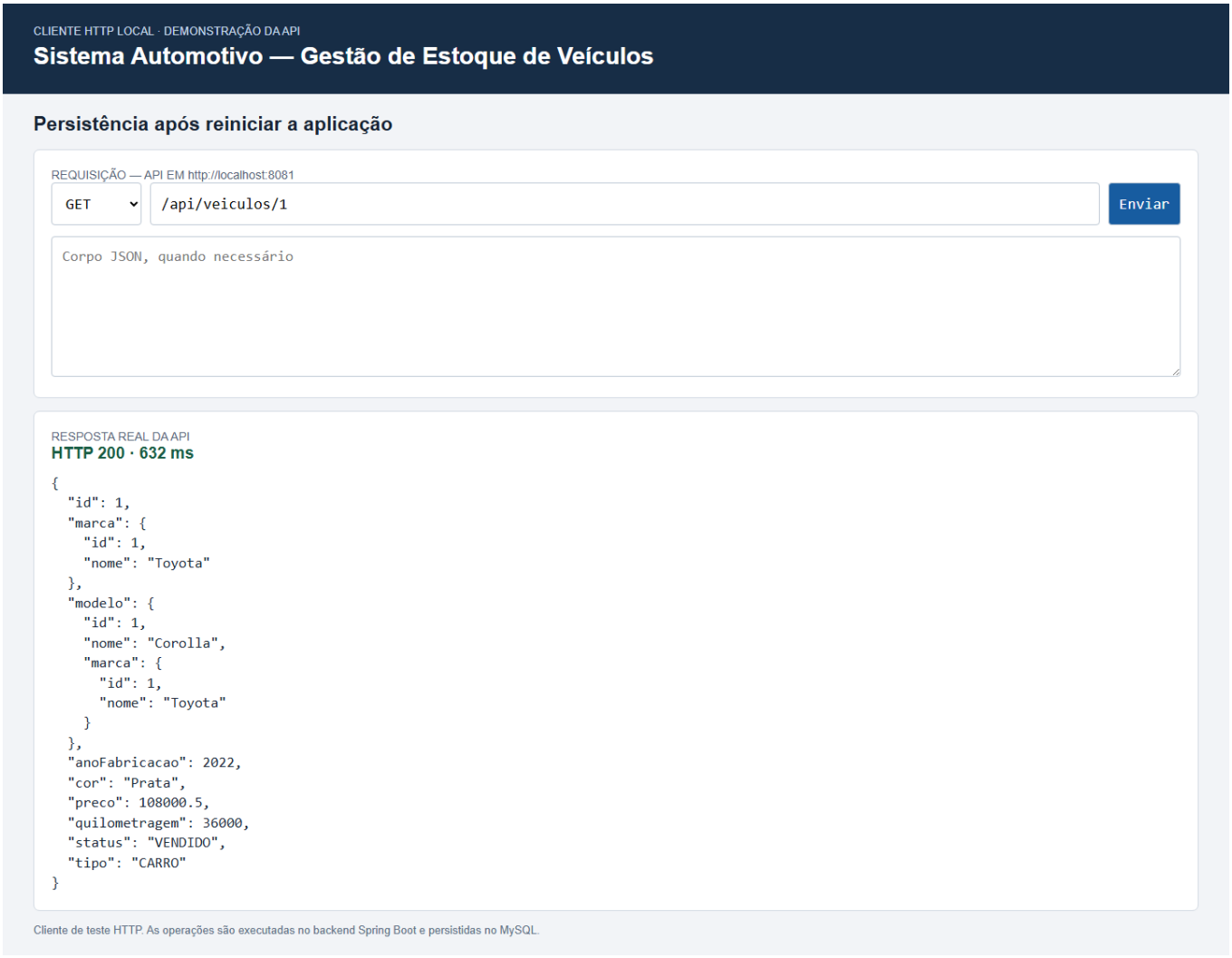
GET /api/veiculos/1
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 15 — Persistência após reiniciar a aplicação

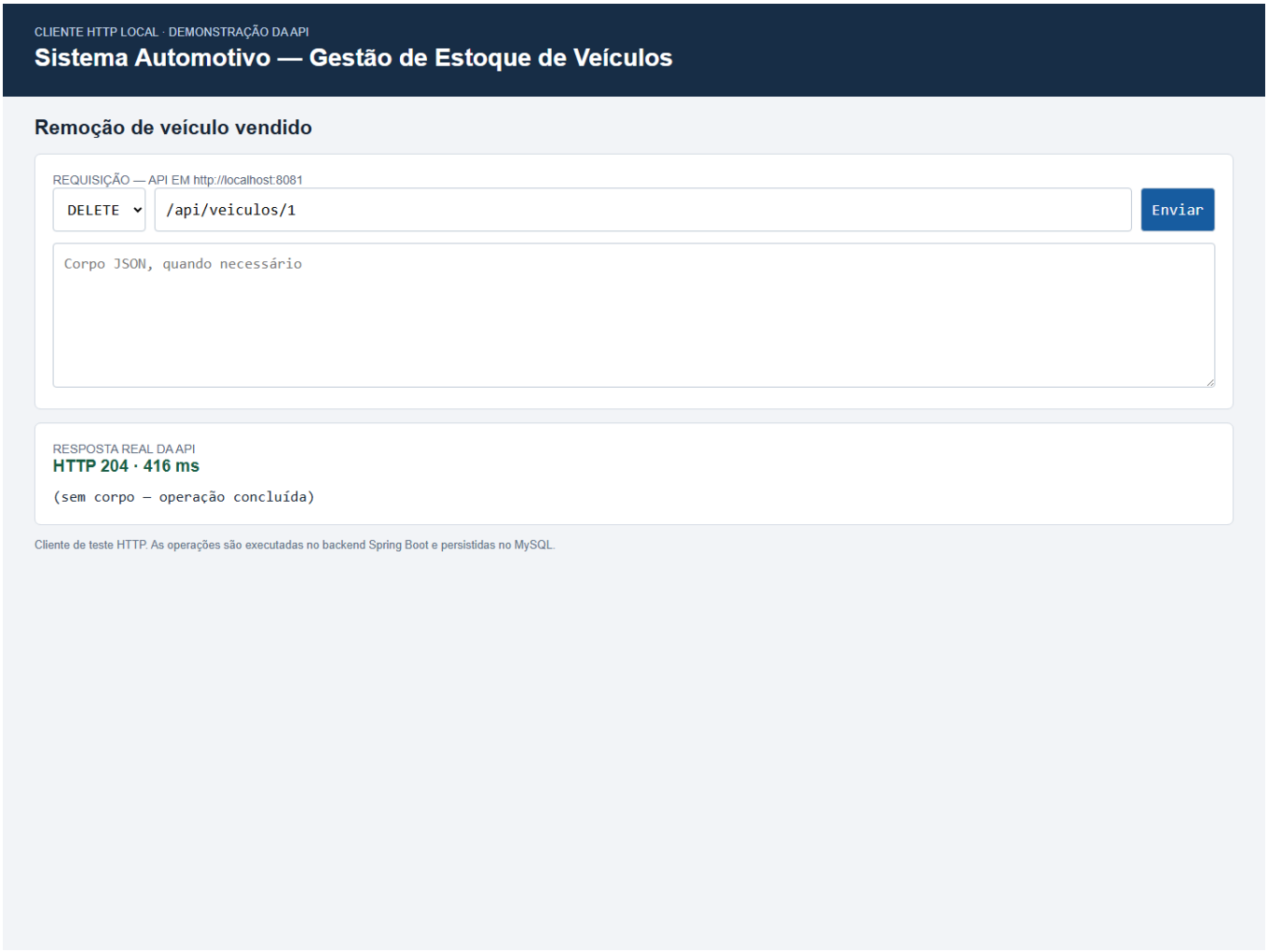
GET /api/veiculos/1
Resultado observado: HTTP 200.



Fonte: captura da execução local da API por cliente HTTP.

Figura 16 — Remoção de veículo vendido

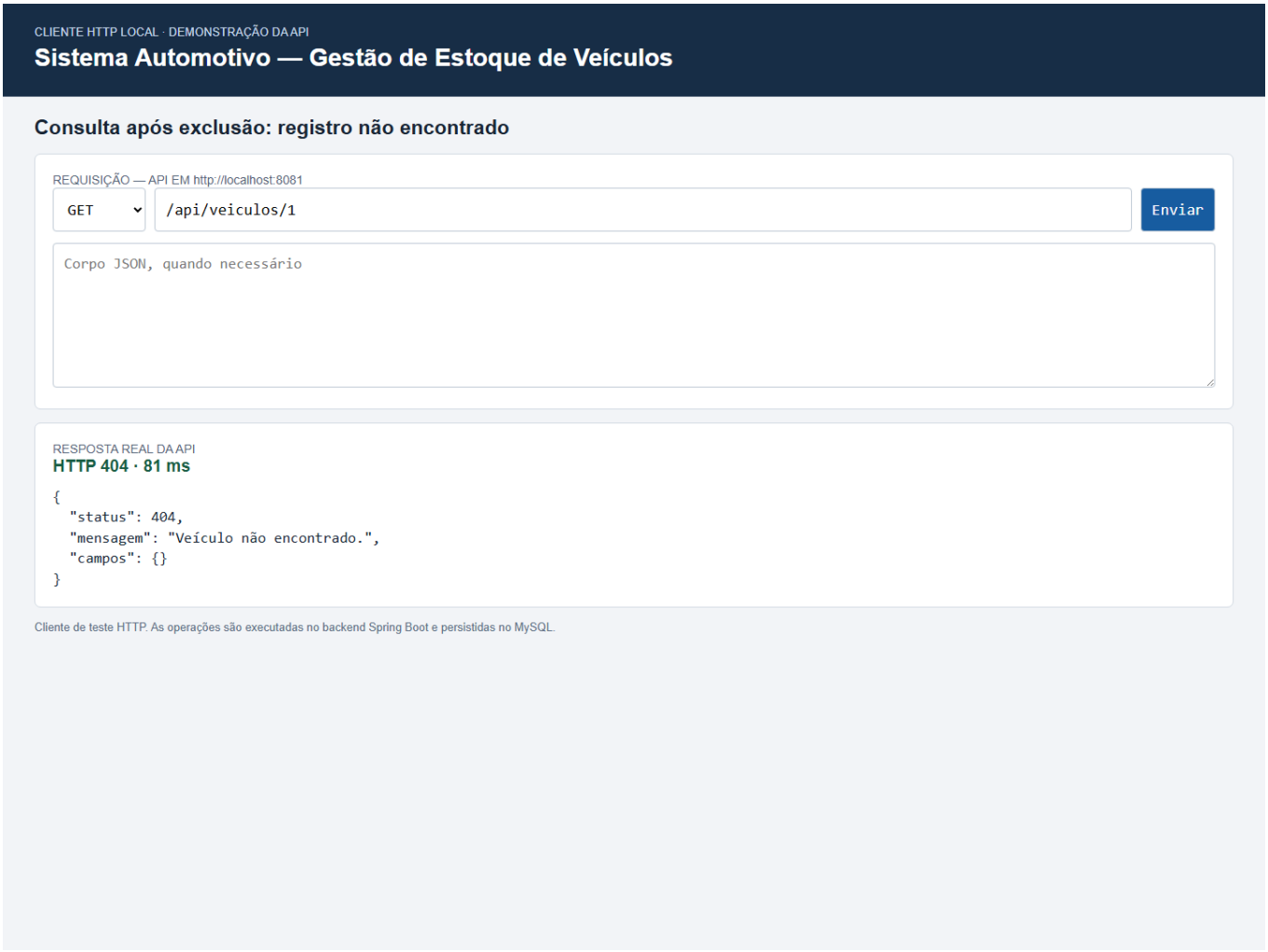
DELETE /api/veiculos/1
Resultado observado: HTTP 204.



Fonte: captura da execução local da API por cliente HTTP.

Figura 17 — Consulta após exclusão: registro não encontrado

GET /api/veiculos/1
Resultado observado: HTTP 404.



Fonte: captura da execução local da API por cliente HTTP.

7. Códigos principais

Os trechos seguintes são extraídos dos arquivos implementados. O código completo, incluindo imports, construtores e arquivos auxiliares, está no repositório público. Os números e quebras visuais do relatório não alteram os arquivos-fonte.

7.1 Herança da identificação

src/main/java/com/unifecaf/automotivo/model/EntidadeBase.java

```
package com.unifecaf.automotivo.model;

import jakarta.persistence.*;

@MappedSuperclass
public abstract class EntidadeBase {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    public Long getId() { return id; }
}
```

7.2 Entidade Veiculo: atributos, associações e encapsulamento

src/main/java/com/unifecaf/automotivo/model/Veiculo.java

```
package com.unifecaf.automotivo.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.Year;
import org.hibernate.annotations.JdbcTypeCode;
import org.hibernate.type.SqlTypes;

@Entity
@Table(name = "veiculo")
public class Veiculo extends EntidadeBase {
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "modelo_id", nullable = false)
    private Modelo modelo;

    @Column(name = "ano_fabricacao", nullable = false)
    private Integer anoFabricacao;
    @Column(nullable = false, length = 50)
    private String cor;
    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal preco;
    @Column(nullable = false)
    private Integer quilometragem;
    @Enumerated(EnumType.STRING)
    @JdbcTypeCode(SqlTypes.VARCHAR)
    @Column(nullable = false, length = 20)
    private StatusVeiculo status;
    @Enumerated(EnumType.STRING)
    @JdbcTypeCode(SqlTypes.VARCHAR)
    @Column(nullable = false, length = 20)
    private TipoVeiculo tipo;

    protected Veiculo() { }

    public Veiculo(Modelo modelo, Integer anoFabricacao, String cor, BigDecimal preco,
                   Integer quilometragem, StatusVeiculo status, TipoVeiculo tipo) {
        if (modelo == null || tipo == null) throw new IllegalArgumentException("Modelo e tipo são
        obrigatórios.");
        if (anoFabricacao == null || anoFabricacao < 1886 || anoFabricacao > Year.now().getValue())
            throw new IllegalArgumentException("Ano de fabricação deve estar entre 1886 e o ano atual.");
        if (cor == null || cor.isBlank() || cor.strip().length() > 50)
            throw new IllegalArgumentException("Informe uma cor de até 50 caracteres.");
        this.modelo = modelo;
        this.anoFabricacao = anoFabricacao;
        this.cor = cor.strip();
        this.tipo = tipo;
        atualizarDadosComerciais(preco, quilometragem, status);
    }

    public void atualizarDadosComerciais(BigDecimal preco, Integer quilometragem, StatusVeiculo status) {
        if (preco == null || preco.signum() <= 0 || preco.compareTo(new BigDecimal("9999999999.99")) > 0
                || preco.stripTrailingZeros().scale() > 2)
            throw new IllegalArgumentException("Preço deve ser positivo, com até 10 inteiros e 2
        decimais.");
        if (quilometragem == null || quilometragem < 0)
            throw new IllegalArgumentException("Quilometragem não pode ser negativa.");
        if (status == null) throw new IllegalArgumentException("Status é obrigatório.");
        this.preco = preco;
        this.quilometragem = quilometragem;
        this.status = status;
    }
```

```
    public boolean podeSerRemovido() { return status == StatusVeiculo.VENDIDO || status ==
        StatusVeiculo.DESCONTINUADO; }
    public Modelo getModelo() { return modelo; }
    public Integer getAnoFabricacao() { return anoFabricacao; }
    public String getCor() { return cor; }
    public BigDecimal getPreco() { return preco; }
    public Integer getQuilometragem() { return quilometragem; }
    public StatusVeiculo getStatus() { return status; }
    public TipoVeiculo getTipo() { return tipo; }
}
```

7.3 Serviço: cadastro e filtros no banco

src/main/java/com/unifecaf/automotivo/service/VeiculoService.java

```
    @Transactional
    public VeiculoSaida cadastrar(VeiculoEntrada dados) {
        Modelo modelo = modelos.findById(dados.modeloId())
                .orElseThrow(() -> new RegraException(NOT_FOUND, "Modelo não encontrado."));
        Veiculo veiculo = new Veiculo(modelo, dados.anoFabricacao(), dados.cor(), dados.preco(),
                dados.quilometragem(), dados.status(), dados.tipo());
        return saida(veiculos.save(veiculo));
    }

    public VeiculoSaida consultar(Long id) { return saida(buscar(id)); }

    public List<VeiculoSaida> listar(Long marcaId, Long modeloId, BigDecimal precoMin,
            BigDecimal precoMax, Integer ano, StatusVeiculo status, TipoVeiculo tipo) {
        if ((precoMin != null && precoMin.signum() < 0) || (precoMax != null && precoMax.signum() < 0))
            throw new IllegalArgumentException("Filtros de preço não podem ser negativos.");
        if (precoMin != null && precoMax != null && precoMin.compareTo(precoMax) > 0)
            throw new IllegalArgumentException("Preço mínimo não pode ser maior que o máximo.");
        Specification<Veiculo> filtro = (root, query, cb) -> {
            List<Predicate> p = new ArrayList<>();
            if (marcaId != null) p.add(cb.equal(root.get("modelo").get("marca").get("id"), marcaId));
            if (modeloId != null) p.add(cb.equal(root.get("modelo").get("id"), modeloId));
            if (precoMin != null) p.add(cb.greaterThanOrEqualTo(root.get("preco"), precoMin));
            if (precoMax != null) p.add(cb.lessThanOrEqualTo(root.get("preco"), precoMax));
            if (ano != null) p.add(cb.equal(root.get("anoFabricacao"), ano));
            if (status != null) p.add(cb.equal(root.get("status"), status));
            if (tipo != null) p.add(cb.equal(root.get("tipo"), tipo));
            query.orderBy(cb.asc(root.get("id")));
            return cb.and(p.toArray(Predicate[]::new));
        };
        return veiculos.findAll(filtro).stream().map(VeiculoService::saida).toList();
    }
```

7.4 Serviço: atualização e remoção

src/main/java/com/unifecaf/automotivo/service/VeiculoService.java

```
    @Transactional
    public VeiculoSaida atualizar(Long id, AtualizacaoVeiculo dados) {
        Veiculo veiculo = buscar(id);
        veiculo.atualizarDadosComerciais(dados.preco(), dados.quilometragem(), dados.status());
        return saida(veiculos.save(veiculo));
    }

    @Transactional
    public void remover(Long id) {
        Veiculo veiculo = buscar(id);
        if (!veiculo.podeSerRemovido())
            throw new RegraException(CONFLICT, "Marque o veículo como vendido ou descontinuado antes de
        removê-lo.");
        veiculos.delete(veiculo);
    }

    private Veiculo buscar(Long id) {
        return veiculos.findById(id).orElseThrow(() -> new RegraException(NOT_FOUND, "Veículo não
        encontrado."));
    }

    private static VeiculoSaida saida(Veiculo v) {
        return new VeiculoSaida(v.getId(), CatalogoService.marcaSaida(v.getModelo().getMarca()),
                CatalogoService.modeloSaida(v.getModelo()), v.getAnoFabricacao(), v.getCor(),
                v.getPreco(), v.getQuilometragem(), v.getStatus(), v.getTipo());
    }
}
```

7.5 Controller REST de veículos

src/main/java/com/unifecaf/automotivo/controller/VeiculoController.java

```
package com.unifecaf.automotivo.controller;

import static com.unifecaf.automotivo.dto.Dados.*;
import com.unifecaf.automotivo.model.*;
import com.unifecaf.automotivo.service.VeiculoService;
import jakarta.validation.Valid;
import java.math.BigDecimal;
import java.net.URI;
import java.util.List;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/veiculos")
public class VeiculoController {
    private final VeiculoService service;
    public VeiculoController(VeiculoService service) { this.service = service; }

    @PostMapping
    public ResponseEntity<VeiculoSaida> cadastrar(@Valid @RequestBody VeiculoEntrada dados) {
        VeiculoSaida veiculo = service.cadastrar(dados);
        return ResponseEntity.created(URI.create("/api/veiculos/" + veiculo.id())).body(veiculo);
    }

    @GetMapping
    public List<VeiculoSaida> listar(@RequestParam(required = false) Long marcaId,
            @RequestParam(required = false) Long modeloId, @RequestParam(required = false) BigDecimal
        precoMin,
            @RequestParam(required = false) BigDecimal precoMax, @RequestParam(required = false) Integer
        ano,
            @RequestParam(required = false) StatusVeiculo status, @RequestParam(required = false)
        TipoVeiculo tipo) {
        return service.listar(marcaId, modeloId, precoMin, precoMax, ano, status, tipo);
    }

    @GetMapping("/{id}")
    public VeiculoSaida consultar(@PathVariable Long id) { return service.consultar(id); }

    @PatchMapping("/{id}")
    public VeiculoSaida atualizar(@PathVariable Long id, @Valid @RequestBody AtualizacaoVeiculo dados) {
        return service.atualizar(id, dados);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> remover(@PathVariable Long id) {
        service.remover(id);
        return ResponseEntity.noContent().build();
    }
}
```

7.6 Contratos de dados e validações

src/main/java/com/unifecaf/automotivo/dto/Dados.java

```
package com.unifecaf.automotivo.dto;

import jakarta.validation.constraints.*;
import java.math.BigDecimal;
import com.unifecaf.automotivo.model.*;

/** Contratos de entrada e saída da API. */
public final class Dados {
    private Dados() { }

    public record MarcaEntrada(
            @NotBlank(message = "Nome é obrigatório.")
            @Size(max = 100, message = "Nome deve ter até 100 caracteres.") String nome) { }

    public record MarcaSaida(Long id, String nome) { }

    public record ModeloEntrada(
            @NotBlank(message = "Nome é obrigatório.")
            @Size(max = 100, message = "Nome deve ter até 100 caracteres.") String nome,
            @NotNull(message = "Marca é obrigatória.") @Positive(message = "ID da marca deve ser
        positivo.") Long marcaId) { }

    public record ModeloSaida(Long id, String nome, MarcaSaida marca) { }

    public record VeiculoEntrada(
            @NotNull(message = "Modelo é obrigatório.") @Positive Long modeloId,
            @NotNull(message = "Ano é obrigatório.") @Min(1886) Integer anoFabricacao,
            @NotBlank(message = "Cor é obrigatória.") @Size(max = 50) String cor,
            @NotNull(message = "Preço é obrigatório.")
            @DecimalMin(value = "0.01", message = "Preço deve ser maior que zero.")
            @Digits(integer = 10, fraction = 2, message = "Preço aceita até 10 inteiros e 2 decimais.")
        BigDecimal preco,
            @NotNull(message = "Quilometragem é obrigatória.")
            @PositiveOrZero(message = "Quilometragem não pode ser negativa.") Integer quilometragem,
            @NotNull(message = "Status é obrigatório.") StatusVeiculo status,
            @NotNull(message = "Tipo é obrigatório.") TipoVeiculo tipo) { }

    public record AtualizacaoVeiculo(
            @NotNull(message = "Preço é obrigatório.")
            @DecimalMin(value = "0.01", message = "Preço deve ser maior que zero.")
            @Digits(integer = 10, fraction = 2, message = "Preço aceita até 10 inteiros e 2 decimais.")
        BigDecimal preco,
            @NotNull(message = "Quilometragem é obrigatória.")
            @PositiveOrZero(message = "Quilometragem não pode ser negativa.") Integer quilometragem,
            @NotNull(message = "Status é obrigatório.") StatusVeiculo status) { }

    public record VeiculoSaida(Long id, MarcaSaida marca, ModeloSaida modelo,
            Integer anoFabricacao, String cor, BigDecimal preco, Integer quilometragem,
            StatusVeiculo status, TipoVeiculo tipo) { }
}
```

7.7 Persistência e carregamento de associações

src/main/java/com/unifecaf/automotivo/repository/VeiculoRepository.java

```
package com.unifecaf.automotivo.repository;

import com.unifecaf.automotivo.model.Veiculo;
import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.domain.Specification;
import org.springframework.data.jpa.repository.*;

public interface VeiculoRepository extends JpaRepository<Veiculo, Long>,
        JpaSpecificationExecutor<Veiculo> {
    @Override
    @EntityGraph(attributePaths = {"modelo", "modelo.marca"})
    List<Veiculo> findAll(Specification<Veiculo> specification);

    @Override
    @EntityGraph(attributePaths = {"modelo", "modelo.marca"})
    Optional<Veiculo> findById(Long id);
}
```

7.8 Cadastro de marcas e modelos

src/main/java/com/unifecaf/automotivo/service/CatalogoService.java

```
package com.unifecaf.automotivo.service;

import static com.unifecaf.automotivo.dto.Dados.*;
import static org.springframework.http.HttpStatus.*;
import com.unifecaf.automotivo.model.*;
import com.unifecaf.automotivo.repository.*;
import com.unifecaf.automotivo.exception.RegraException;
import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional(readOnly = true)
public class CatalogoService {
    private final MarcaRepository marcas;
    private final ModeloRepository modelos;

    public CatalogoService(MarcaRepository marcas, ModeloRepository modelos) {
        this.marcas = marcas;
        this.modelos = modelos;
    }

    @Transactional
    public MarcaSaida cadastrarMarca(MarcaEntrada dados) {
        if (marcas.existsByNomeIgnoreCase(dados.nome().strip()))
            throw new RegraException(CONFLICT, "Marca já cadastrada.");
        return marcaSaida(marcas.save(new Marca(dados.nome())));
    }

    public List<MarcaSaida> listarMarcas() { return
        marcas.findAll().stream().map(CatalogoService::marcaSaida).toList(); }

    @Transactional
    public ModeloSaida cadastrarModelo(ModeloEntrada dados) {
        Marca marca = marcas.findById(dados.marcaId())
                .orElseThrow(() -> new RegraException(NOT_FOUND, "Marca não encontrada."));
        if (modelos.existsByNomeIgnoreCaseAndMarcaId(dados.nome().strip(), marca.getId()))
            throw new RegraException(CONFLICT, "Modelo já cadastrado para essa marca.");
        return modeloSaida(modelos.save(new Modelo(dados.nome(), marca)));
    }

    public List<ModeloSaida> listarModelos() { return
        modelos.findAll().stream().map(CatalogoService::modeloSaida).toList(); }
    static MarcaSaida marcaSaida(Marca m) { return new MarcaSaida(m.getId(), m.getNome()); }
    static ModeloSaida modeloSaida(Modelo m) { return new ModeloSaida(m.getId(), m.getNome(),
        marcaSaida(m.getMarca())); }
}
```

7.9 Modelo físico MySQL

database/schema.sql

```
CREATE TABLE IF NOT EXISTS marca (
    id BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    CONSTRAINT uk_marca_nome UNIQUE (nome)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS modelo (
    id BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    marca_id BIGINT NOT NULL,
    CONSTRAINT uk_modelo_marca_nome UNIQUE (marca_id, nome),
    CONSTRAINT fk_modelo_marca FOREIGN KEY (marca_id) REFERENCES marca(id)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS veiculo (
    id BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    modelo_id BIGINT NOT NULL,
    ano_fabricacao INT NOT NULL,
    cor VARCHAR(50) NOT NULL,
    preco DECIMAL(12,2) NOT NULL,
    quilometragem INT NOT NULL,
    status VARCHAR(20) NOT NULL,
    tipo VARCHAR(20) NOT NULL,
    CONSTRAINT fk_veiculo_modelo FOREIGN KEY (modelo_id) REFERENCES modelo(id),
    CONSTRAINT ck_preco CHECK (preco > 0),
    CONSTRAINT ck_quilometragem CHECK (quilometragem >= 0),
    CONSTRAINT ck_status CHECK (status IN ('DISPONIVEL','VENDIDO','DESCONTINUADO')),
    CONSTRAINT ck_tipo CHECK (tipo IN ('CARRO','MOTO','UTILITARIO')),
    INDEX idx_preco (preco),
    INDEX idx_ano (ano_fabricacao),
    INDEX idx_status (status)
) ENGINE=InnoDB;
```

8. Reprodução e entrega

O README do repositório contém as instruções para iniciar o banco por Docker ou utilizar MySQL instalado, definir DB_USER/DB_PASSWORD/DB_URL, iniciar pelo Maven Wrapper e executar os testes. A coleção em postman/ contém o fluxo de cadastro, consultas, filtros, atualização e exclusão.

O teste automatizado utiliza exclusivamente o banco estoque_veiculos_test e limpa seus registros entre cenários. Os dados do estoque normal não são usados pelos testes. O modelo físico está em database/schema.sql; o banco deve existir antes de iniciar a API.

Código-fonte: https://github.com/igorguimaraes-dev/sistema-automotivo-poo.

Vídeo pitch: https://youtu.be/nzAiVEuqOuE. Arquivo local: pitch.mp4. Informe esse link no campo de entrega da atividade.

9. Limitações do escopo

Não foi criado frontend, por ser opcional. As operações são demonstradas por cliente HTTP. O teste de desempenho possui base pequena e execução local. Não foram adicionados login, pagamentos, cadastro de clientes, relatórios financeiros ou outras funcionalidades não solicitadas.